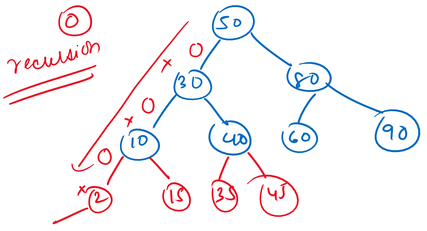
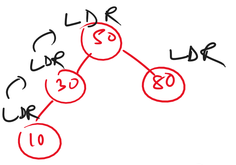
Binary Search Tree :→ Most Important Operations :→

1. insert (root, data)
2. search (root, data)
3. delete (root, data) {*** imp for technical viva rounds}

For all these operations we either go left or right, so we actually skip approximately half of the tree every time.

So, average time complexity is Big O (log N) just like Binary Search. [sorted array → O(N)]





o/p 10 30 50 80

* How do you check whether a tree is a BST?

⇒ InOrder Traversal (LDR) :→ Ascending Order

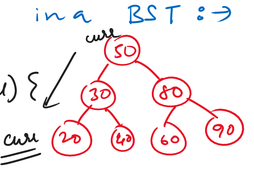
2 10 15 30 35 40 45 50 60 80 90

* Find the minimum node of a subtree in a BST :→

Floor

```
FindMin(node) {
    Node curr = node;
    while(curr != null && curr.left != null) {
        curr = curr.left;
    }
    return curr;
}
```

findMin(50)

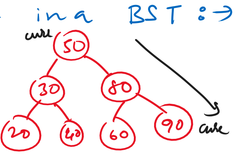


* Find the maximum node of a subtree in a BST :→

Ceil

```
FindMax(node) {
    Node curr = node;
    while(curr != null && curr.right != null) {
        curr = curr.right;
    }
    return curr;
}
```

findMax(50)

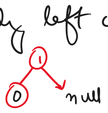


* Delete Operation :→

Case :→ Node with only one child

(a) Only left child

BST



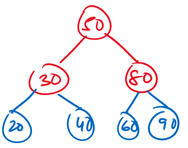

```
if (root.right == null) {
    Tree temp = r.l
    root = null;
    return temp;
}
```

(b) Only right child

BST

```
if (root.right == null) {
    Tree temp = r.r
    root = null;
    return temp;
}
```

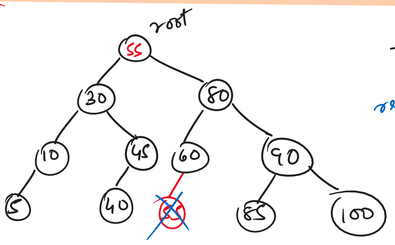
Normal Case



delete (20) Left

delete (90) Right

Case : Delete a node with both children :→



delete (root, 50)

```
TreeNode temp = findMin(root.right)
              = findMin(80)
              = 55;
{ root.data = temp.data; }
delete(root.right, temp.data);
```

replace root with (55)

delete the duplicate

InOrder :→

5 10 30 40 45 50 55 60 80 85 90 100

(Too many confusions) Inorder Predecessor (Successor) (Leaf value of RHS)